SOC Alerts Dashboard

Project Report

Author: Rahul Jha
Date: May 10, 2026
Version: 1.0.0

Executive Summary

SOC Alerts Dashboard is a production-grade, full-stack triage application engineered to help security analysts authenticate securely, inspect real-time alert trends, filter massive queues, and update alert statuses efficiently. The system utilizes a decoupled architecture with isolated frontend and backend services, a well-defined REST API contract, and persistent, relational storage via PostgreSQL.

1 Engineering Approach

Product & Workflow

Prioritized the critical path: Login → Dashboard → Queue → Detail.

Maintained sub-second perceived load times via pagination and debounced search.

Preserved application state in URL parameters for highly shareable, reproducible views.

Architecture & Security

Implemented strict domain-based routing (`/api/auth`, `/api/alerts`).

Secured sessions via `httpOnly` JWT cookies, preventing XSS token theft.

Centralized query helpers ensure clean filter construction and defend against SQLi.

Technology Stack & Runtime

Frontend (Client)

- React 18 & Vite
- React Router v6
- Tailwind CSS
- Axios

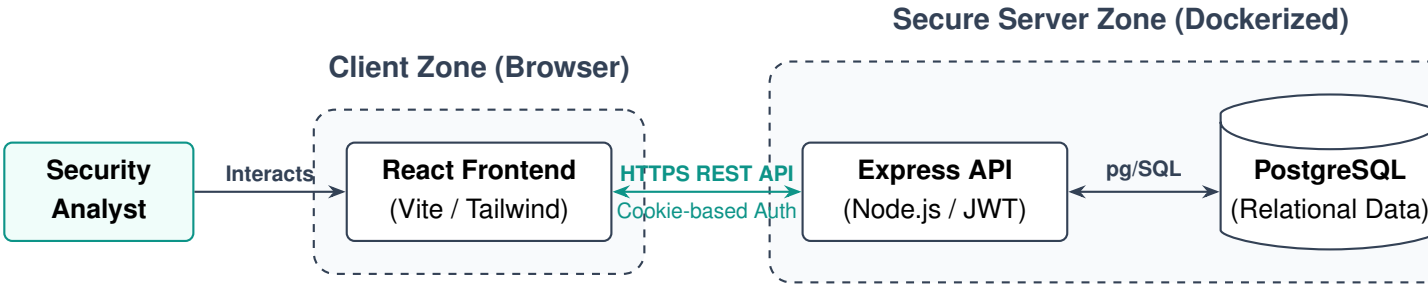
Backend (API)

- Node.js & Express.js
- PostgreSQL (`pg`)
- JSON Web Tokens
- bcryptjs

Infrastructure

- Docker / Compose
- Nginx Reverse Proxy
- `.env` Configuration
- Vitest (Testing)

2 System Architecture



3 REST API Contract

All endpoints are versioned and exposed under the protected `/api` base path.

Authentication

POST `/api/auth/login`
Authenticate & set `httpOnly` cookie.

POST `/api/auth/logout`
Terminate session & clear cookie.

GET `/api/auth/me`
Validate JWT and return user profile.

Alert Management

GET `/api/alerts`
Fetch paginated alerts (supports filters).

GET `/api/alerts/:id`
Retrieve detailed alert telemetry.

PATCH `/api/alerts/:id`
Update triage status, severity, or assignee.

GET `/api/alerts/stats/dashboard`
Return aggregated metrics for charts.

4 Deployment & Quick Start

To initialize the development environment with live-reloading, execute the following in two isolated terminal instances from the project root:

Execution Commands

```
# Terminal 1:  Initialize Database & Backend API
$ cd backend
$ npm install
$ npm run dev

# Terminal 2:  Initialize Client Interface
$ cd frontend
$ npm install
$ npm run dev
```